

Bodegas

Pepe Calderón, Fernando Sola, Daniel Ayala, Ana Belén Sánchez, Inma Hernández, Margarita Cruz, Carlos Arévalo, David Ruiz



Escuela Técnica Superior de
Ingeniería Informática

Índice

1. Requisitos de información (RI)	2
1.1. RI-1: Bodega	2
1.2. RI-2: Vino	2
1.3. RI-3: Uvas	3
1.4. RI-4: Cosechas	3
2. Reglas de negocio (RN)	4
2.1. RN-1: Unicidad	4
2.2. RN-2: Tiempo de crianza	4
2.3. RN-3: Graduación alcohólica	4
2.4. RN-4: Cosechas	5
3. Requisitos funcionales (RF)	6
3.1. RF-1: Informes simples de Bodegas	6
4. Pruebas de aceptación (PA)	7
4.1. PA-1: Bodegas	7
4.2. PA-2: Vinos	7
5. Diagrama de clases	9
6. Posible extensión	10
7. Álgebra relacional	13
8. Script SQL para crear la base de datos	18
9. Script SQL para la carga inicial de datos	21
10. Consultas	23
11. Pruebas SQL	25

Requisitos

Requisitos de información (RI)

1.1. RI-1: Bodega

- Como: Profesor de la asignatura
- Quiero: Poder almacenar la siguiente información de las bodegas: nombre y denominación de origen.
- Para: Que el estudiante realice a partir de este requisito el modelo conceptual, relacional y tecnológico.

1.2. RI-2: Vino

- Como: Profesor de la asignatura
- Quiero: Poder almacenar la siguiente información de los vinos: nombre, grados de alcohol y Uvas que lo componen. Los vinos pueden ser de Jóvenes o Crianzas, en función al tiempo que pasen en bodega y botella. Además, para los vinos Crianzas, se debe almacenar las cosechas.
- Para: Que el estudiante realice a partir de este requisito el modelo conceptual, relacional y tecnológico.

1.3. RI-3: Uvas

- Como: Profesor de la asignatura
- Quiero: Poder almacenar la siguiente información de las uvas: nombre (Tempranillo, Garnacha, Carbernet Sauvignon, ...)
- Para: Que el estudiante realice a partir de este requisito el modelo conceptual, relacional y tecnológico.

1.4. RI-4: Cosechas

- Como: Profesor de la asignatura
- Quiero: Poder almacenar la siguiente información de las cosechas: año y calidad (Excelente, Muy buena, Buena, Normal, Mala)
- Para: Que el estudiante realice a partir de este requisito el modelo conceptual, relacional y tecnológico.

Reglas de negocio (RN)

2.1. RN-1: Unicidad

- Como: Profesor de la asignatura
- Quiero: Que no se repitan el nombre de las bodegas, vinos o uvas en el sistema
- Para: Que el estudiante practique con restricciones simples.

2.2. RN-2: Tiempo de crianza

- Como: Profesor de la asignatura
- Quiero: Que el tiempo de crianza de un vino Crianza sea de al menos 24 meses, de los cuales entre 6 y 12 meses son en barrica y el resto en botella. El vino joven generalmente no pasa tiempo en barrica, o si lo hace, es muy breve (menos de 6 meses)
- Para: Que el estudiante practique con restricciones simples.

2.3. RN-3: Graduación alcohólica

- Como: Profesor de la asignatura
- Quiero: Que la graduación alcohólica de un vino suele estar entre los 10 y 15 grados
- Para: Que el estudiante practique con restricciones complejas.

2.4. RN-4: Cosechas

- Como: Profesor de la asignatura
- Quiero: Cada vino de crianza tiene como máximo una cosecha por año, y tiene al menos una cosecha
- Para: Que el estudiante practique con restricciones complejas.

Requisitos funcionales (RF)

3.1. RF-1: Informes simples de Bodegas

- Como: Profesor de la asignatura
- Quiero: Seleccionar todas las bodegas con denominación de origen Rioja. Listado de vinos con sus uvas. Crianzas con sus cosechas. Mostrar todas las bodegas que producen vinos tanto jóvenes como crianzas. Nombre de las bodegas y vinos que están compuestos, al menos, con uva 'Tempranillo'. Total de crianzas por cosecha. Nombre del vino joven con más grados
- Para: Que el estudiante practique con consultas simples.

Pruebas de aceptación (PA)

4.1. PA-1: Bodegas

1. ☒ Crear una nueva bodega con todos los datos correctos según las reglas de negocio
2. ☐ Crear una nueva bodega sin nombre.
3. ☐ Crear una nueva bodega con el nombre repetido.
4. ☐ Crear una nueva bodega sin denominación de origen.

4.2. PA-2: Vinos

1. ☒ Crear un nuevo vino Joven con todos los datos correctos según las reglas de negocio
2. ☒ Crear un nuevo vino Crianza con todos los datos correctos según las reglas de negocio
3. ☐ Crear un nuevo vino Joven sin nombre.
4. ☐ Crear un nuevo vino Crianza sin nombre.
5. ☐ Crear un nuevo vino Joven con el nombre repetido.
6. ☐ Crear un nuevo vino Crianza con el nombre repetido.
7. ☐ Crear un nuevo vino Joven sin grados de alcohol.
8. ☐ Crear un nuevo vino Crianza sin grados de alcohol.
9. ☐ Crear un nuevo vino Joven con graduación incorrecta.
10. ☐ Crear un nuevo vino Crianza con graduación incorrecta.
11. ☐ Crear un nuevo vino Joven sin bodega.
12. ☐ Crear un nuevo vino Crianza sin bodega.

Modelo Conceptual

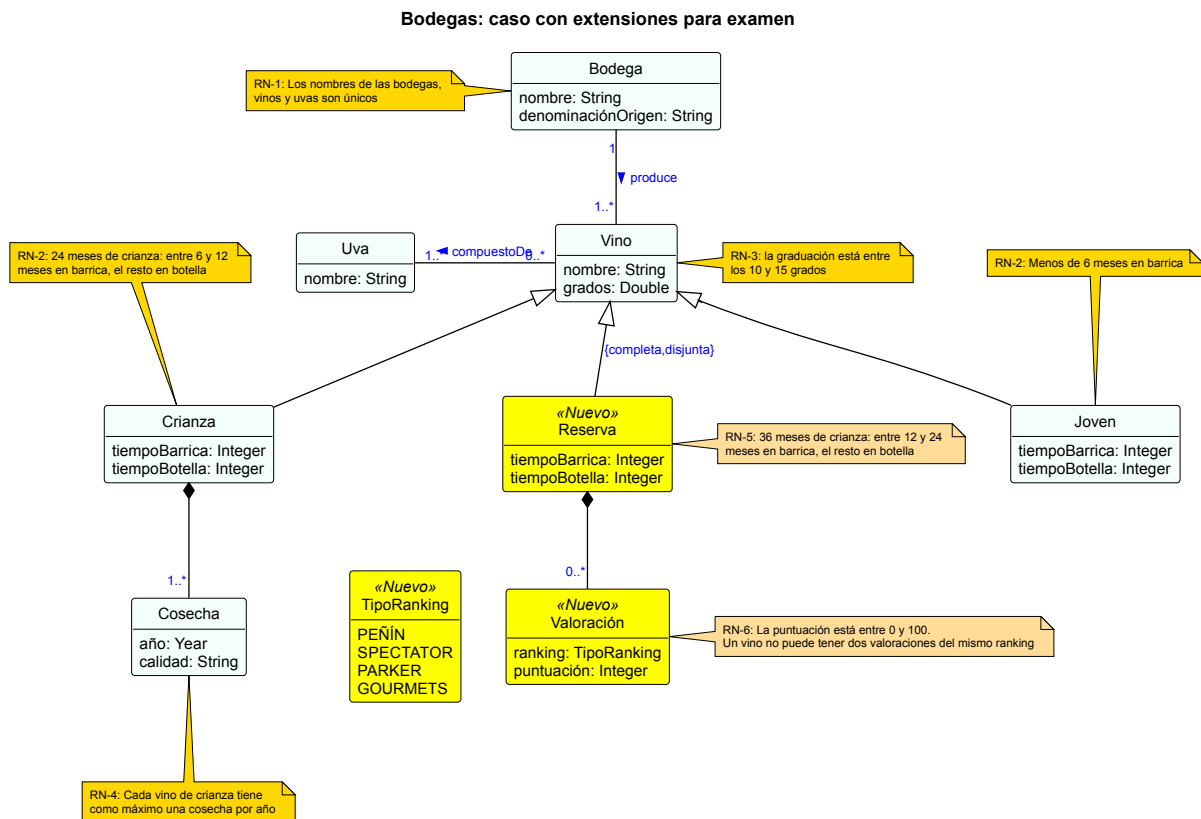
```
classDiagram
    class Bodega {
        nombre: String
        denominación: String
    }
    class Vino {
        nombre: String
        grados: Double
    }
    class Joven {
        tiempoBarrica: Integer
        tiempoBotella: Integer
    }
    class Crianza {
        tiempoBarrica: Integer
        tiempoBotella: Integer
    }
    class Cosecha {
        año: Year
        calidad: String
    }
    class Uva {
        nombre: String
    }

    Bodega "1" -- "1..*" Vino : produce
    Joven --|> Vino : {completa,disjunta}
    Crianza --|> Joven
    Cosecha "1..*" -- "0..*" Crianza
    Uva "1..*" -- "0..*" Crianza : elaboradoCon
```

Posible extensión

- Jerarquía extendida: Vino se especializa en Joven, Crianza y Reserva {completa, disjunta}.
- Reserva compone 0..* Valoración (ranking: TipoRanking, puntuación 0..100); no puede haber dos valoraciones del mismo ranking para un mismo vino (RN-6).
- Se mantienen Bodega–Vino (1..), *Crianza–Cosecha* (1..), y Vino–Uva (M:N) como en el modelo base.
- RN-5: Reserva 36 meses (12–24 en barrica; resto en botella). Además, se mantienen RN-1..RN-4.

6. Posible extensión



Modelo Relacional

```

Bodegas = { bodegaId, nombre, denominaciónOrigen }
    PK(bodegaId)
    AK(nombre)
Vinos = { vinoId, bodegaId, nombre, grados }
    PK(vinoId)
    FK(bodegaId) / Bodegas
    AK(nombre)
Jóvenes = { vinoId, tiempoBarrica, tiempoBotella }
    PK(vinoId)
    FK(vinoId) / Vinos
Crianzas = { vinoId, tiempoBarrica, tiempoBotella }
    PK(vinoId)
    FK(vinoId) / Vinos
Uvas = { uvaId, nombre }
    PK(uvaId)
    AK(nombre)
Cosechas = { cosechaId, crianzaId, año, calidad }
    PK(cosechaId)
    FK(crianzaId) / Crianzas
    AK(cosechaId, crianzaId, año)
VinosUvas = { vinoUvaId, vinoId, uvaId }
    PK(vinoUvaId)
    FK(vinoId) / Vinos
    FK(uvaId) / Uvas
    AK(vinoId, uvaId)
Bodegas = {
    (b1, "Bodegas El Sol", "Rioja"),
    (b2, "Bodegas La Luna", "Ribera del Duero")
}
Vinos = {
    (v1, b1, "Vino Blanco Joven", 12),

```

```

        (v2, b2, "Vino Tinto Joven", 13),
        (v3, b1, "Vino Crianza Especial", 14),
        (v4, b2, "Vino Crianza Reserva", 13.5)
    }
    Jóvenes = { (v1, 0, 6), (v2, 0, 12) }
    Crianzas = { (v3, 6, 18), (v4, 12, 12) }
    Uvas = {
        (u1, "Tempranillo"),
        (u2, "Garnacha"),
        (u3, "Albarino")
    }
    Cosechas = {
        (c1, v3, 2020, "Excelente"),
        (c2, v3, 2019, "Buena"),
        (c3, v4, 2018, "Muy buena")
    }
    VinosUvas = {
        (vu1, v1, u3),
        (vu2, v2, u1),
        (vu3, v3, u2),
        (vu4, v3, u1),
        (vu5, v4, u2),
        (vu6, v4, u1)
    }
}

```

Álgebra relacional

-Renombrado:

$$B = \rho_{B(bid,nb,do)} (Bodegas)$$

$$V = \rho_{V(vid,bid,nv,g)} (Vinos)$$

$$J = \rho_{J(vid,tba,tbo)} (Jóvenes)$$

$$C = \rho_{C(vid,tba,tbo)} (Crianzas)$$

$$U = \rho_{U(uid,nu)} (Uvas)$$

$$Co = \rho_{Co(coid,vid,a,c)} (Cosechas)$$

$$VU = \rho_{VU(vuid,vid,uid)} (VinosUvas)$$

- Seleccionar todas las bodegas con denominación de origen Rioja:

$$Riojas \leftarrow \sigma_{do = \text{“Rioja”}} (B)$$

Riojas = { bid, nb, do }

Riojas = {

```
(b1, "Bodegas El Sol", "Rioja")
}
```

- Listado de vinos con sus uvas:

$$VVUU \leftarrow V \bowtie VU \bowtie U$$

```
VVUU = {vid, bid, nv, g, vuid, uid, nu}

VVUU = {
  (v1, b1, "Vino Blanco Joven", 12, vu1, u3, "Albarino"),
  (v2, b2, "Vino Tinto Joven", 13, vu2, u1, "Tempranillo"),
  (v3, b1, "Vino Crianza Especial", 14, vu3, u2, "Garnacha"),
  (v3, b1, "Vino Crianza Especial", 14, vu4, u1, "Tempranillo"),
  (v4, b2, "Vino Crianza Reserva", 13.5, vu5, u2, "Garnacha"),
  (v4, b2, "Vino Crianza Reserva", 13.5, vu6, u1, "Tempranillo")
}
```

- Crianzas con sus cosechas:

$$CCo \leftarrow V \bowtie C \bowtie Co$$

```
CCo = {vid, bid, nv, g, tba, tbo, coid, a, c}

CCo = {
  (v3, b1, "Vino Crianza Especial", 14, 6, 18, c1, 2020, "Excelente"),
  (v3, b1, "Vino Crianza Especial", 14, 6, 18, c2, 2019, "Buena"),
  (v4, b2, "Vino Crianza Reserva", 13.5, 12, 12, c3, 2018, "Muy buena")
}
```

- Mostrar todas las bodegas que producen vinos tanto jóvenes como crianzas:

$$BV \leftarrow B \bowtie V$$

```
BV = { bid, nb, do, vid, nv, g }

BV = {
  (b1, "Bodegas El Sol", "Rioja", v1, "Vino Blanco Joven", 12),
  (b2, "Bodegas La Luna", "Ribera del Duero", v2, "Vino Tinto Joven", 13),
  (b1, "Bodegas El Sol", "Rioja", v3, "Vino Crianza Especial", 14),
  (b2, "Bodegas La Luna", "Ribera del Duero", v4, "Vino Crianza Reserva", 13.5)
}
```

$$BodegasJovenes \leftarrow \Pi_{bid,nb} (BV \bowtie J)$$

```
BodegasJovenes = { bid, nb }

BodegasJovenes = {
  (b1, "Bodegas El Sol"),
  (b2, "Bodegas La Luna")
}
```

$$BodegasCrianzas \leftarrow \Pi_{bid,nb} (BV \bowtie C)$$

7. Álgebra relacional

```
BodegasCrianzas = { bid, nb }
```

```
BodegasCrianzas = {
  (b1, "Bodegas El Sol"),
  (b2, "Bodegas La Luna")
}
```

$$BodegasJovenes \cap BodegasCrianzas$$

```
BodegasJovenesCrianzas = { bid, nb }
```

```
BodegasJovenesCrianzas = {
  (b1, "Bodegas El Sol"),
  (b2, "Bodegas La Luna")
}
```

- Nombre de las bodegas y vinos que están compuestos, al menos, con uva "Tempranillo":

$$BT \leftarrow \Pi_{nb, nv} \left(\sigma_{nu = \text{"Tempranillo"}} (BV \bowtie VU \bowtie U) \right)$$

```
BT = { nb, nv }
```

```
BT = {
  ("Bodegas La Luna", "Vino Tinto Joven"),
  ("Bodegas El Sol", "Vino Crianza Especial"),
  ("Bodegas La Luna", "Vino Crianza Reserva")
}
```

- Total de cosechas por vino de crianza:

$$TotalCosechasCrianza \leftarrow \gamma_{vid, \rho_{total}(\text{COUNT}(*))} vid(C \bowtie Co)$$

```
TotalCosechasCrianza = { vid, total }
```

```
TotalCosechasCrianza = {
  (v3, 2),
  (v4, 1)
}
```

- Nombre del vino joven con más grados:

$$maxGrados \leftarrow \gamma_{\rho_{maxG}(\text{MAX}(g))} (V \bowtie J)$$

```
maxGrados = { maxG }
```

```
maxGrados = {
  (13)
}
```

$$VinoMasGrados \leftarrow \Pi_{nv} \left(\sigma_{g = maxG} ((V \bowtie J) \times maxGrados) \right)$$

```
VinoMasGrados = { nv }
```

```
VinoMasGrados = {  
  ("Vino Tinto Joven")  
}
```

- Número de vinos crianza por cosecha:

$$NumCCo \leftarrow \frac{a, \rho_{total}(COUNT(vid))}{\gamma} a(Co)$$

```
NumCCo = { a, total }
```

```
NumCCo = {  
  (2018, 1),  
  (2019, 1),  
  (2020, 1)  
}
```

- Bodegas con más vinos:

$$BodegasNumVinos \leftarrow \frac{bid, nb, \rho_{total}(COUNT(vid))}{\gamma} bid, nb(BV)$$

```
BodegasNumVinos = { bid, nb, total }
```

```
BodegasNumVinos = {  
  (b1, "Bodegas El Sol", 2),  
  (b2, "Bodegas La Luna", 2)  
}
```

$$maxVinos \leftarrow \frac{\rho_{maxTotal}(MAX(total))}{\gamma} (BodegasNumVinos)$$

```
maxVinos = { maxTotal }
```

```
maxVinos = {  
  (2)  
}
```

$$BodegasMasVinos \leftarrow \prod_{bid, nb} \left(\frac{\sigma}{total=maxTotal} (BodegasNumVinos \times maxVinos) \right)$$

```
BodegasMasVinos = { bid, nb }
```

```
BodegasMasVinos = {  
  (b1, "Bodegas El Sol"),  
  (b2, "Bodegas La Luna")  
}
```

- Vinos que tienen, al menos, las mismas uvas que el vino 'v1':

$$VinosUvasV1 \leftarrow \prod_{vid, nv} \left(\left(\prod_{vid, uid} (VU) \div \prod_{uid} \left(\frac{\sigma}{vid=v1} (VU) \right) \right) \bowtie V \right)$$

7. Álgebra relacional

```
VinosUvasV1 = { vid, nv }  
  
VinosUvasV1 = {  
    (v1, "Vino Blanco Joven")  
}
```

Modelo Tecnológico

Script SQL para crear la base de datos

bodegas/createDB.sql

```
--
-- Autor: David Ruiz
-- Fecha: Noviembre 2024
-- Descripción: Script para crear la BD de Bodegas
--

DROP DATABASE IF EXISTS BodegasDB;
CREATE DATABASE BodegasDB;
USE BodegasDB;

-- =====
-- Eliminamos tablas previas (orden inverso de dependencias)
-- =====

SET FOREIGN_KEY_CHECKS = 0;
DROP TABLE IF EXISTS wine_grapes;
DROP TABLE IF EXISTS harvests;
DROP TABLE IF EXISTS young_wines;
DROP TABLE IF EXISTS aged_wines;
DROP TABLE IF EXISTS wines;
DROP TABLE IF EXISTS grapes;
DROP TABLE IF EXISTS wineries;
SET FOREIGN_KEY_CHECKS = 1;
```

```

-- =====
-- Tabla: wineries
-- =====
CREATE TABLE wineries (
    winery_id INT AUTO_INCREMENT,
    winery_name VARCHAR(255) NOT NULL,
    origin_designation VARCHAR(255) NOT NULL,
    PRIMARY KEY (winery_id),
    CONSTRAINT rn01_wineries_unique_name UNIQUE (winery_name)
);

-- =====
-- Tabla: wines
-- =====
CREATE TABLE wines (
    wine_id INT AUTO_INCREMENT,
    winery_id INT NOT NULL,
    wine_name VARCHAR(255) NOT NULL,
    alcohol_percent DECIMAL(4,2) NOT NULL,
    PRIMARY KEY (wine_id),
    FOREIGN KEY (winery_id) REFERENCES wineries(winery_id),
    CONSTRAINT rn01_wines_unique_name UNIQUE (wine_name),
    CONSTRAINT rn03_wines_alcohol_range CHECK (alcohol_percent BETWEEN 10
AND 15)
);

-- =====
-- Tabla: young_wines
-- =====
CREATE TABLE young_wines (
    wine_id INT,
    barrel_months TINYINT NOT NULL,
    bottle_months TINYINT NOT NULL,
    PRIMARY KEY (wine_id),
    FOREIGN KEY (wine_id) REFERENCES wines(wine_id),
    CONSTRAINT rn02_young_wines_age CHECK (
        barrel_months <= 6 AND barrel_months + bottle_months <= 12
    )
);

-- =====
-- Tabla: aged_wines
-- =====
CREATE TABLE aged_wines (
    wine_id INT,
    barrel_months TINYINT NOT NULL,
    bottle_months TINYINT NOT NULL,
    PRIMARY KEY (wine_id),
    FOREIGN KEY (wine_id) REFERENCES wines(wine_id),
    CONSTRAINT rn02_aged_wines_age CHECK (

```

```

        barrel_months >= 6 AND barrel_months + bottle_months >= 24
    )
);

-- =====
-- Tabla: grapes
-- =====
CREATE TABLE grapes (
    grape_id INT AUTO_INCREMENT,
    grape_name VARCHAR(255) NOT NULL,
    PRIMARY KEY (grape_id),
    CONSTRAINT rn01_grapes_unique_name UNIQUE (grape_name)
);

-- =====
-- Tabla: harvests
-- =====
CREATE TABLE harvests (
    harvest_id INT AUTO_INCREMENT,
    wine_id INT NOT NULL,
    harvest_year YEAR NOT NULL,
    quality VARCHAR(100) NOT NULL,
    PRIMARY KEY (harvest_id),
    FOREIGN KEY (wine_id) REFERENCES wines(wine_id) ON DELETE CASCADE,
    CONSTRAINT rn04_harvests_unique_year UNIQUE (wine_id, harvest_year)
);

-- =====
-- Tabla: wine_grapes
-- =====
CREATE TABLE wine_grapes (
    wine_id INT,
    grape_id INT,
    PRIMARY KEY (wine_id, grape_id),
    CONSTRAINT fk_wine_grapes_wine FOREIGN KEY (wine_id) REFERENCES
wines(wine_id),
    CONSTRAINT fk_wine_grapes_grape FOREIGN KEY (grape_id) REFERENCES
grapes(grape_id)
);

```

Script SQL para la carga inicial de datos

bodegas/populateDB.sql

```
--  
-- Autor: David Ruiz  
-- Fecha: Noviembre 2024  
-- Descripción: Procedimiento para poblar la BD de Bodegas  
--  
USE BodegasDB;  
  
DELIMITER //  
CREATE OR REPLACE PROCEDURE p_populate()  
BEGIN  
    SET FOREIGN_KEY_CHECKS = 0;  
    DELETE FROM wine_grapes;  
    DELETE FROM harvests;  
    DELETE FROM young_wines;  
    DELETE FROM aged_wines;  
    DELETE FROM wines;  
    DELETE FROM grapes;  
    DELETE FROM wineries;  
    SET FOREIGN_KEY_CHECKS = 1;  
  
    INSERT INTO wineries (winery_id, winery_name, origin_designation)  
VALUES  
    (1, 'Bodegas El Sol', 'Rioja'),
```

```

        (2, 'Bodegas La Luna', 'Ribera del Duero');

INSERT INTO wines (wine_id, winery_id, wine_name, alcohol_percent)
VALUES
    (1, 1, 'Vino Blanco Joven', 12.0),
    (2, 2, 'Vino Tinto Joven', 13.0),
    (3, 1, 'Vino Crianza Especial', 14.0),
    (4, 2, 'Vino Crianza Reserva', 13.5);

INSERT INTO young_wines (wine_id, barrel_months, bottle_months) VALUES
    (1, 0, 6),
    (2, 0, 12);

INSERT INTO aged_wines (wine_id, barrel_months, bottle_months) VALUES
    (3, 6, 18),
    (4, 12, 12);

INSERT INTO grapes (grape_id, grape_name) VALUES
    (1, 'Tempranillo'),
    (2, 'Garnacha'),
    (3, 'Albarino');

INSERT INTO harvests (harvest_id, wine_id, harvest_year, quality)
VALUES
    (1, 3, 2020, 'Excelente'),
    (2, 3, 2019, 'Buena'),
    (3, 4, 2018, 'Muy buena');

INSERT INTO wine_grapes (wine_id, grape_id) VALUES
    (1, 3),
    (2, 1),
    (3, 2),
    (3, 1),
    (4, 2),
    (4, 1);

END //
DELIMITER ;

CALL p_populate();

```

Consultas

bodegas/queries.sql

```
--
-- Autor: David Ruiz
-- Fecha: Noviembre 2024
-- Descripción: Consultas de referencia para la BD de Bodegas
--

USE BodegasDB;

-- Bodegas con denominación Rioja
SELECT *
FROM wineries
WHERE origin_designation = 'Rioja';

-- Listado de vinos con sus uvas
SELECT w.wine_name, g.grape_name
FROM wines w
JOIN wine_grapes wg ON w.wine_id = wg.wine_id
JOIN grapes g ON wg.grape_id = g.grape_id;

-- Vinos con sus cosechas
SELECT w.*, h.harvest_year, h.quality
FROM wines w
JOIN harvests h ON w.wine_id = h.wine_id;

-- Bodegas que producen vinos jóvenes y crianzas
SELECT DISTINCT wy.winery_name
```

```

FROM wineries wy
JOIN wines w ON wy.winery_id = w.winery_id;

-- Bodegas y vinos que usan la uva Tempranillo
SELECT wy.winery_name, w.wine_name
FROM wineries wy
JOIN wines w ON wy.winery_id = w.winery_id
JOIN wine_grapes wg ON w.wine_id = wg.wine_id
JOIN grapes g ON wg.grape_id = g.grape_id
WHERE g.grape_name = 'Tempranillo';

-- Total de cosechas por vino de crianza
SELECT h.wine_id, COUNT(*) AS total_harvests
FROM harvests h
JOIN aged_wines aw ON h.wine_id = aw.wine_id
GROUP BY h.wine_id;

-- Vino joven con más grados
SELECT w.wine_name, w.alcohol_percent
FROM wines w
JOIN young_wines yw ON w.wine_id = yw.wine_id
ORDER BY w.alcohol_percent DESC
LIMIT 1;

-- Conteo de cosechas por vino
SELECT w.wine_name, COUNT(*) AS total_harvests
FROM wines w
JOIN harvests h ON w.wine_id = h.wine_id
GROUP BY w.wine_name;

```

Pruebas SQL

bodegas/tests.sql

```
--
-- Autor: David Ruiz
-- Fecha: Noviembre 2024
-- Descripción: Tests negativos para la BD de Bodegas
--
USE BodegasDB;

-- =====
-- TABLA DE RESULTADOS
-- =====
CREATE OR REPLACE TABLE test_results (
    test_id VARCHAR(20) PRIMARY KEY,
    test_name VARCHAR(200) NOT NULL,
    test_message VARCHAR(500) NOT NULL,
    test_status ENUM('PASS', 'FAIL', 'ERROR') NOT NULL,
    execution_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- =====
-- PROCEDIMIENTO AUXILIAR
-- =====
DELIMITER //
CREATE OR REPLACE PROCEDURE p_log_test(
    IN p_test_id VARCHAR(20),
    IN p_message VARCHAR(500),
    IN p_status ENUM('PASS', 'FAIL', 'ERROR')
```

```

)
BEGIN
    INSERT INTO test_results(test_id, test_name, test_message, test_status)
    VALUES (p_test_id, SUBSTRING_INDEX(p_message, ':', 1), p_message,
p_status);
END //
DELIMITER ;

-- =====
-- TESTS (RN01 - RN04)
-- =====

DELIMITER //
CREATE OR REPLACE PROCEDURE p_test_rn01_unique_winery()
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
        CALL p_log_test('RN01', 'RN01: Los nombres de bodegas deben ser
únicos', 'PASS');

    CALL p_populate();

    INSERT INTO wineries (winery_id, winery_name, origin_designation)
    VALUES (10, 'Bodegas El Sol', 'Rioja');

    CALL p_log_test('RN01', 'ERROR: Se permitió duplicar el nombre de una
bodega', 'FAIL');
END //
DELIMITER ;

DELIMITER //
CREATE OR REPLACE PROCEDURE p_test_rn02_young_limits()
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
        CALL p_log_test('RN02', 'RN02: Los vinos jóvenes respetan los
límites de meses', 'PASS');

    CALL p_populate();

    INSERT INTO wines (wine_id, winery_id, wine_name, alcohol_percent)
    VALUES (20, 1, 'Vino Joven Invalido', 12.0);

    INSERT INTO young_wines (wine_id, barrel_months, bottle_months)
    VALUES (20, 8, 10);

    CALL p_log_test('RN02', 'ERROR: Se permitió un vino joven con
demasiados meses en barrica', 'FAIL');
END //
DELIMITER ;

DELIMITER //

```

```

CREATE OR REPLACE PROCEDURE p_test_rn03_alcohol_range()
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
        CALL p_log_test('RN03', 'RN03: La graduación alcohólica debe estar
entre 10 y 15', 'PASS');

    CALL p_populate();

    INSERT INTO wines (wine_id, winery_id, wine_name, alcohol_percent)
    VALUES (30, 1, 'Vino Fuera Rango', 9.0);

    CALL p_log_test('RN03', 'ERROR: Se permitió una graduación alcohólica
fuera de rango', 'FAIL');
END //
DELIMITER ;

DELIMITER //
CREATE OR REPLACE PROCEDURE p_test_rn04_harvest_per_year()
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
        CALL p_log_test('RN04', 'RN04: Solo una cosecha por vino y año',
'PASS');

    CALL p_populate();

    INSERT INTO harvests (harvest_id, wine_id, harvest_year, quality)
    VALUES (10, 3, 2020, 'Duplicada');

    CALL p_log_test('RN04', 'ERROR: Se permitió duplicar cosecha de un vino
para el mismo año', 'FAIL');
END //
DELIMITER ;

-- =====
-- ORQUESTADOR
-- =====
DELIMITER //
CREATE OR REPLACE PROCEDURE p_run_tests()
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
        BEGIN
            CALL p_log_test('POPULATE', 'ERROR: El populate falló. No se
ejecutaron los tests negativos.', 'ERROR');
            SELECT * FROM test_results ORDER BY execution_time, test_id;
            SELECT test_status, COUNT(*) AS total FROM test_results GROUP BY
test_status;
        END;

    DELETE FROM test_results;

```

```
CALL p_populate();

CALL p_test_rn01_unique_winery();
CALL p_test_rn02_young_limits();
CALL p_test_rn03_alcohol_range();
CALL p_test_rn04_harvest_per_year();

SELECT * FROM test_results ORDER BY execution_time, test_id;
SELECT test_status, COUNT(*) AS total FROM test_results GROUP BY
test_status;
END //
DELIMITER ;

CALL p_run_tests();
```

i Info

Versión PDF disponible